Reviewer Pack — Minimal Rank of Geometric Quantization of Boolean Functions …

Entry 7882afae52ae · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 05:23:43 UTC

Minimal Rank of Geometric Quantization of Boolean Functions over Tseitin Circuit Width

Entry ID: `7882afae52ae` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 05:23:43 UTC

1. Conjecture

Field A (mathematical branch): Geometric Quantization **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

> {'quantitative_relation': 'E[rank(GeometricQuantization(F))] = Θ(w(Tseitin(F)))', 'falsifier_formulation': 'There exists a CNF F such that rank(GeometricQuantization(F)) < w(Tseitin(F))'}

Rationale (proposer's reasoning):

> {'explanation1': 'Geometric quantization transforms the configuration space of a system into a symplectic manifold, which may capture non-local properties of boolean functions.', 'explanation2': 'Tseitin circuits are a simple and natural model for boolean functions, making them amenable to geometric analysis.', 'explanation3': 'The conjecture suggests that the geometric quantization rank provides a lower bound on Tseitin circuit width.'}

Taxonomy category: `Geometric_Quantization` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `019c3c596ce796e5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If the mean rank of GeometricQuantization over Tseitin circuit width does not significantly deviate from the expected value for at least 80% of the seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `title:geometric quantization AND tseitin circuit width` - `quantum complexity AND geometric quantization of boolean functions` - `arithmetic complexity AND tseitin circuit width AND geometric quantization`

Top relevant hits considered: - [http://arxiv.org/abs/1902.08753v2] Quantum Learning Boolean Linear Functions w.r.t. Product Distributions - [http://arxiv.org/abs/2406.18700v4] Structure of sparse Boolean functions over Abelian groups, and its application to testing - [http://arxiv.org/abs/2511.23445v2] Quantum Polymorphisms and the Complexity of Quantum Constraint Satisfaction - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/0706.3841v1] Arithmetic lattices and weak spectral geometry - [http://arxiv.org/abs/1605.04207v1] Functional lower bounds for arithmetic circuits and connections to boolean circuit complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = [random.choice(variables) if random.choice([True, False]) else -random.choice
            clauses.append(clause)
        return clauses

    def tseitin_width(cnf):
        # Simplified Tseitin width calculation
        return len(set(abs(lit) for lit in cnf))

    def geometric_quantization_rank(cnf):
        # Placeholder for actual geometric quantization rank computation
        # For simplicity, we use a dummy function that returns the number of variables
        return len(cnf)

    n = random.randint(5, 40)
```

```
    m = random.randint(n * 2, n * 3)
    cnf = generate_cnf(n, m)
    width = tseitin_width(cnf)
    rank = geometric_quantization_rank(cnf)

    return {
        "metric_name": "Rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank >= width,
        "counterexample": "" if rank >= width else f"CNF with n={n}, m={m} has rank {rank} < width
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=NA support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"rank < width\" first_failing_seed={first_failin
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b16c0a4.py", line 57, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b16c0a4.py", line 41, in run_trial
    width = tseitin_width(cnf)
            ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b16c0a4.py", line 31, in tseitin_width
    return len(set(abs(lit) for lit in cnf))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b16c0a4.py", line 31, in <genexpr>
    return len(set(abs(lit) for lit in cnf))
                   ^^^^^^^^
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed during execution, which prevents us from evaluating the conjecture's support or falsification. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11484
2	propose	ollama_remote	glm4:latest	0	0	10096
3	preregistration	ollama_remote	glm4:latest	0	0	5676
4	novelty	ollama_remote	glm4:latest	0	0	4564
5	novelty	ollama_remote	glm4:latest	0	0	6290
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13009
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9647
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7975
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6977
10	judge	ollama_remote	glm4:latest	0	0	9747

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 85465 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/7882afae52ae.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7882afae52ae.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7882afae52ae.tar.gz` (if generated)